

How remember() and recall() move a fact from conversation to ranked answer

Alice mentions a coffee preference once. Weeks later, a different query about her habits still surfaces it — ranked, scored, and typed — without any conversation ID or session state carried by the caller.

USER_ID = "ALICE"

"I prefer oat milk in my coffee"

"I work as a software engineer at TechCorp"

"I want to learn ML this year"

01_basic_remember_recall.py

from hippocampai import MemoryClient

client = MemoryClient()

client.remember(text="I prefer oat milk...", user_id="alice", type="preference", importance=8.0,)

MEMORYCLIENT — WRITE PATH

1 • Classify

infer type if not given — fact / preference / goal / habit / event

2 • Embed

BAAI/bge-small-en-v1.5 → 384-dim vector

3 • Store

write to Qdrant, tag user_id + importance + half-life

recall("coffee preferences")

Read path

embed query → hybrid rank (see poster 02) → top-k

QDRANT VECTOR STORE

Durable • scoped by user_id • no session or conversation ID required

persists across sessions

RANKED RECALL — "HOW DOES ALICE LIKE HER COFFEE?"

"I prefer oat milk in my coffee"

score 0.91

preference

"I work as a software engineer..."

score 0.18

fact

No conversation ID to thread

Memory is keyed by user_id, not a session — it survives across every future conversation automatically.

✓

Type-aware decay

Preferences/goals decay over 90 days; events over 14 — see 15_hippocampus_ai/00_concepts.

☆

Score, not just match

Every result carries a breakdown of the signals that produced it — detailed in the next poster.

Four signals, one ranked list: how recall() actually scores a memory

A single embedding-similarity number is not enough to rank memories — recency and stated importance matter as much as semantic closeness. Every candidate is scored on all four signals before the list is returned.

📄 QUERY — "WHAT ARE CAROL'S DIETARY RESTRICTIONS?"

VECTOR SIMILARITY

× 0.55

cosine match against the query embedding — top ~200 candidates from Qdrant

CROSS-ENCODER RERANK

× 0.20

ms-marco-MiniLM-L-6-v2 pairwise relevance, precision pass

RECENCY DECAY

× 0.15

$\exp(-\Delta t / \text{half-life})$ — type-specific decay rate

STATED IMPORTANCE

× 0.10

the 0–10 score set at `remember()` time

→ **Σ weighted fusion** →

1. "I am lactose intolerant"

fact · importance 9.0

final 0.87 — sim .81 · rerank .93 · recency .77 · importance .90
2. "I am allergic to peanuts"

fact · importance 10.0

final 0.79 — sim .68 · rerank .84 · recency .64 · importance 1.0
3. "I prefer vegetarian food and avoid meat"

preference · importance 8.0

final 0.52 — sim .58 · rerank .41 · recency .70 · importance .80

☆ **Similarity alone would rank differently**
Peanut allergy has the highest raw similarity to "dietary restrictions" — rerank and importance are what push lactose intolerance above it here.

≡ **BM25 runs in parallel, not shown**
A keyword pass catches exact-term matches embeddings under-rank; both feed reciprocal rank fusion before this stage.

🕒 **Weights are configuration, not code**
`WEIGHT_SIM` / `WEIGHT_RERANK` / `WEIGHT_RECENCY` / `WEIGHT_IMPORTANCE` in `.env`.

What happens while the agent "sleeps": a night's memories, replayed and compressed

The direct computational analogue of hippocampal replay — a nightly pass that decides what today's raw, redundant memories become tomorrow's durable knowledge.

1

Collect — last 24 hours, per user

Query for event / context memories created or updated since the last run.

- "Had coffee at 9am"
- "Meeting at 10am"
- "Discussed Q4 roadmap"

2

Cluster — group by session, time, and semantic proximity

The three memories above collapse into one cluster: "morning_routine".

3

LLM review — one call per cluster

The model reads the cluster and returns a structured verdict, not free text:

```
{ "promoted_facts": [{ "id": "3", "reason": "strategic discussion" }], "low_value_memory_ids": ["1"], "synthetic_memories": [{ "text": "Morning: productive session, Q4 planning", "importance": 7.0, "source_ids": ["2","3"] }] }
```

4

Apply decisions

Low-value memories are archived, promoted facts get an importance bump, and the synthesized summary is written as a new, durable memory.

- 1 archived
- 1 promoted
- 1 synthesized

5

Log the run

A ConsolidationRun record and Prometheus metrics — this poster's row 4 outcome is exactly what gets logged, per user, per night.

 **Off by default in production**
AUTO_CONSOLIDATION_ENABLED=false — it costs one LLM call per cluster, per user, per night.

 **Nothing is deleted blind**
"Low value" memories are archived with provenance, not hard-deleted — the synthetic memory still cites its source IDs.

Three architecture families, side by side

Not a ranking — a fit question. Each family optimizes a different production requirement, and the field's own mid-2026 guidance is to test against your workload rather than pick from a leaderboard.

	OS-INSPIRED TIERED Letta · MemGPT lineage	TEMPORAL KNOWLEDGE GRAPH Zep · Graphiti	HYBRID VECTOR + GRAPH + KV Mem0 · HippocampAI
Core unit	Core / recall / archival tiers — agent pages memory in and out, like an OS manages RAM/cache/disk	Bi-temporal graph edges with t_valid / t_invalid — facts are invalidated, never deleted	Short extracted facts across three complementary indexes, fused at query time
Manages memory	Agent-driven — the model calls memory functions itself as part of its reasoning	Framework-driven — ingestion pipeline extracts and links automatically	Framework-driven — write/read path handled by the client library
Reported signal	Strong on long-horizon, multi-session continuity	63.8% LongMemEval (GPT-4o) vs. 49.0% for a hybrid baseline — leading reported temporal-reasoning score; 94.8% vs. 93.4% (MemGPT) on Deep Memory Retrieval	Single-digit-ms write latency; fastest bolt-on time-to-value
Best fit	Long-running stateful agents that must reason about <i>why</i> they're forgetting something	Systems of record: "what was true when," contradiction resolution, relationship traversal	Fast personalization on an existing agent; full self-hosted control incl. consolidation
Everyday analogue	RAM (core) → L2/L3 cache (recall) → disk (archival)	A ledger that keeps every superseded entry, timestamped, instead of overwriting it	Three specialist indexes (semantic / relational / exact-match) voting on one ranked answer

^① Benchmarks per vendor/framework self-reporting as of mid-2026 (LoCoMo, LongMemEval, BEAM) — independent production testing shows a real gap from these numbers; treat as directional, not a leaderboard. See